

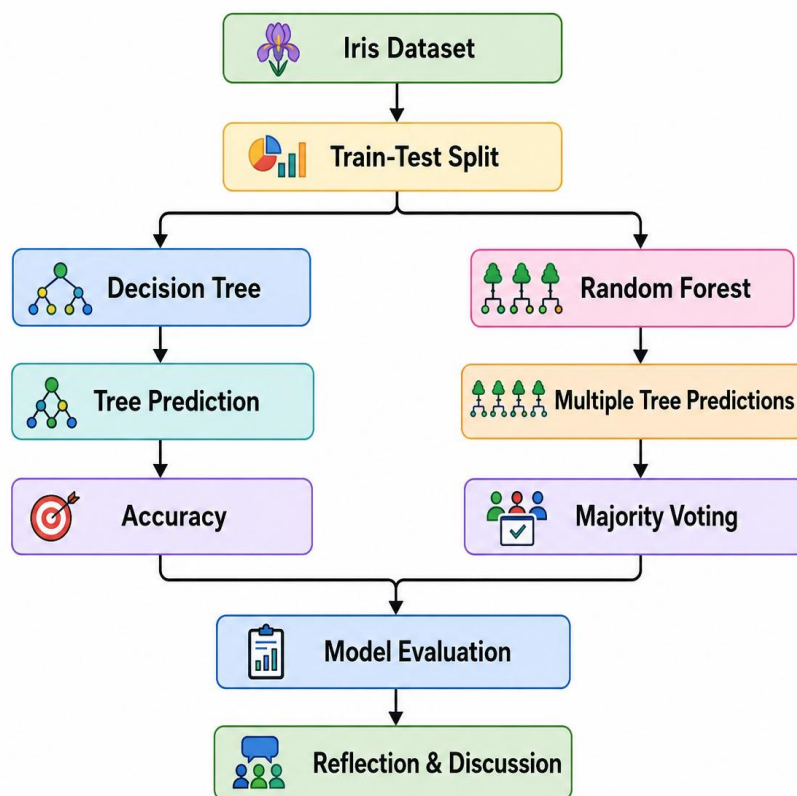
DECISION TREES & RANDOM FOREST

“Build Our Own Forest” – Day 10 Task

Name: Manish Giri | **Reg. No:** 137256 | **Email:** manishgiri8101@gmail.com | [\[LinkedIn\]](#)

Task Workflow Overview

The diagram below illustrates the complete pipeline for this task: from loading the Iris dataset through Train-Test Split, running a Decision Tree and Random Forest in parallel, and finally combining results for Model Evaluation and Reflection.



Notebook Goal: Build Our Own Forest

The primary goal of this notebook is to understand and implement Decision Trees and Random Forests using the Iris dataset. We will:

- Load and explore the Iris dataset
- Split data into training and testing sets
- Train a Decision Tree classifier and evaluate accuracy
- Demonstrate overfitting with a deep tree
- Build a Random Forest and compare performance
- Analyze feature importance

Pseudocode — Decision Tree Algorithm

```
START

Load Iris Dataset

Split dataset into Train and Test

Create Decision Tree

WHILE stopping condition not reached
    Select best feature
    Split data
    Create child nodes
END WHILE

Predict test samples

Compute Accuracy

STOP
```

Part 1: Decision Tree Challenge

We train a Decision Tree with `max_depth=3` on the Iris dataset, make predictions, and compute accuracy.

Code — Decision Tree Classifier

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

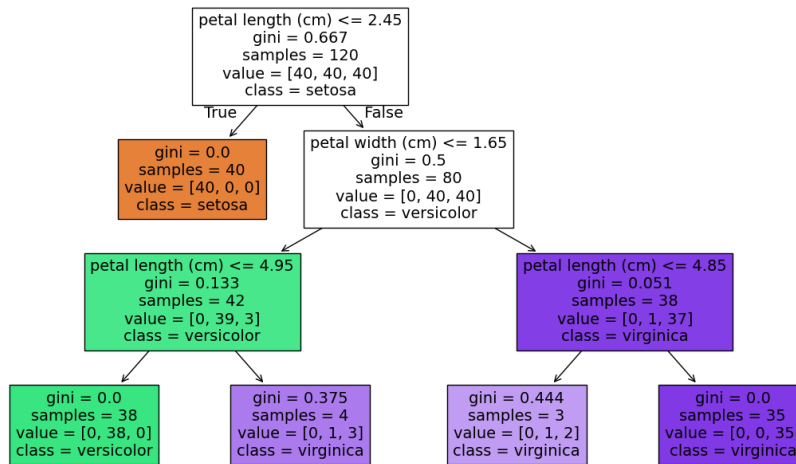
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
tree_model = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_model.fit(X_train, y_train)
tree_preds = tree_model.predict(X_test)
tree_acc = accuracy_score(y_test, tree_preds)
print('Decision Tree Accuracy:', round(tree_acc, 4))
```

Output — Decision Tree Accuracy: 0.9667

Decision Tree —



Overfitting Demonstration

A very deep tree (no max_depth limit) may memorize training data and generalize poorly to new data:

```
deep_tree = DecisionTreeClassifier(random_state=42)
deep_tree.fit(X_train, y_train)
train_acc = accuracy_score(y_train, deep_tree.predict(X_train))
test_acc = accuracy_score(y_test, deep_tree.predict(X_test))
# Result: Training Accuracy: 1.0 | Testing Accuracy: ~0.9333
```

Output —

Deep Tree Training Accuracy: 1.0

Deep Tree Testing Accuracy: 0.9333

The deep tree achieves 100% on training data but drops to ~93% on test data — a clear sign of overfitting.

Part 2: Random Forest Simulation

A Random Forest trains multiple trees and combines their predictions via majority voting, reducing overfitting and improving robustness.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100, max_depth=3, random_state=42
)
rf_model.fit(X_train, y_train)
```

```
rf_preds = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, rf_preds)
print('Random Forest Accuracy:', round(rf_acc, 4))
```

Output — Random Forest Accuracy: 0.9667

Feature Importance (from Random Forest)

The Random Forest can rank features by their contribution to predictions:

Feature	Importance (approx.)
petal length (cm)	0.4412
petal width (cm)	0.4193
sepal length (cm)	0.0970
sepal width (cm)	0.0425

Classification Report –

Class	Precision	Recall	F1-Score	Support
Setosa	1.00	1.00	1.00	10
Versicolor	1.00	0.90	0.95	10
Virginica	0.91	1.00	0.95	10
Accuracy			0.97	30
Macro Avg	0.97	0.97	0.97	30
Weighted Avg	0.97	0.97	0.97	30

WORKSHEET: DATA CLEANSING WITH DECISION TREES

Section 1 – Spot the Dirty Data

Worksheet: Data Cleansing with Decision Trees

Section 1 – Spot the Dirty Data

Task: Identify issues in the sample dataset below.

Passenger	Sex	Age	Fare	Survived
1	male	?	512	0
2	Female	25	71	1
3	M	8	21	1
4	male	35	?	0

Reflection: What kinds of problems do you see? (Missing values, inconsistent entries, outliers)

Section 2 – Tree-Based Cleansing

Worksheet: Data Cleansing with Decision Trees

Section 2 – Tree-Based Cleansing

Task: Use a Decision Tree to guide corrections.

- Split on Sex, Age, and Fare.
- Predict missing values or flag outliers.

Feature	Split Condition	Cleansing Action
Sex	male vs Female	Standardize labels
Age	< 10	Impute missing ages
Fare	> 300	Flag as outlier

Reflection: How do tree splits help reveal data issues?

Section 3 – Random Forest Robustness

Worksheet: Data Cleansing with Decision Trees

Section 3 – Random Forest Robustness

Task: Discuss how multiple trees handle noisy data.

- Each tree uses different subsets of features.
- Majority vote reduces the effect of errors.

Reflection: Why is ensemble learning more reliable for messy datasets?

Section 4 – Summary

- **Decision Tree:** Detects patterns and anomalies.
- **Random Forest:** Smooths out noise and improves accuracy.
- **Outcome:** Cleaner, more trustworthy data for modeling.

ANSWERS TO WORKSHEET SECTIONS

Section 1 Answer – Spot the Dirty Data

Reflection: What kinds of problems do you see? (Missing values, inconsistent entries, outliers)

Looking at the sample dataset from the worksheet:

Passenger	Sex	Age	Fare	Survived
1	male	?	512	0
2	Female	25	71	1
3	M	8	21	1
4	male	35	?	0

Identified Problems:

- Missing Values: Passenger 1 has Age = '?' and Passenger 4 has Fare = '?' — these are null/missing entries that must be imputed or removed before modeling.
- Inconsistent Labels: The 'Sex' column uses three different representations for the same concept: 'male', 'Female', and 'M'. These should all be standardized to one format (e.g., 'male' / 'female').
- Outlier: Passenger 1 has a Fare of 512, which is far above the other fares (21, 71). This is a statistical outlier and may skew model learning.

Example with Titanic Data:

In a real Titanic dataset, Age has about 20% missing values. A common fix is to impute the missing ages using the median age of passengers in the same passenger class (Pclass). For example, if a 3rd-class passenger has no age recorded, we fill it with the median age of all 3rd-class passengers. Fare outliers above 300 are typically flagged and either capped or treated separately.

Section 2 Answer – Tree-Based Cleansing

Reflection: How do tree splits help reveal data issues?

A Decision Tree splits data based on feature thresholds to separate classes. These splits naturally highlight where data behaves unexpectedly — which is exactly where data issues hide.

Feature	Split Condition	Cleansing Action
Sex	male vs Female/M	Standardize labels (all lowercase)
Age	< 10	Impute missing ages (e.g., with median)
Fare	> 300	Flag as outlier for special treatment

How Splits Reveal Issues — Explained:

- Sex Column: When the tree tries to split on 'Sex', it sees three distinct values: 'male', 'Female', and 'M'. This forces the tree to create extra branches for what should be one category, reducing efficiency and accuracy.
- Age Column: When the tree splits on Age < 10, passengers with missing Age ('?') cannot be routed left or right — they fall into an undefined branch. This makes the model unreliable for those rows.
- Fare Column: When Fare > 300 is tested, Passenger 1 (Fare=512) gets isolated into its own branch. This single outlier can skew the entire subtree, causing the model to overfit to one anomalous data point.

Example:

Imagine a tree splitting on Fare > 300. If only 1 out of 100 passengers has Fare > 300, that branch represents just 1% of data. The tree dedicates a full split to one noisy point, wasting model complexity. By flagging and capping such outliers before training, we allow the tree to find more generalizable patterns.

Section 3 Answer – Random Forest Robustness

Reflection: Why is ensemble learning more reliable for messy datasets?

A single Decision Tree is sensitive to noise: one outlier or missing value can change the entire tree structure. A Random Forest builds many trees, each on a different random subset of data and features. Errors made by individual trees get 'outvoted' by the majority.

Two Key Mechanisms:

- Each tree uses different subsets of features: If one tree is misled by the noisy Fare outlier (512), another tree that did not use Fare at all will still make a correct prediction. Feature randomness spreads the risk.
- Majority vote reduces errors: The final class prediction is the class chosen by most trees. If 80 out of 100 trees correctly predict 'Setosa', the one or two trees confused by dirty data cannot change the outcome.

Example from the Notebook (Majority Voting):

```
# 5 trees make predictions on the same sample:  
tree_predictions = ['Setosa', 'Setosa', 'Versicolor', 'Setosa', 'Setosa']  
# Tree 3 is wrong (Versicolor), but majority wins:  
Final Prediction = 'Setosa' (4 votes vs 1)
```

In this example, Tree 3 may have been trained on a subset that included the noisy Fare=512 outlier, causing it to misclassify. But the other 4 trees, trained on cleaner subsets, correctly predict 'Setosa', and the majority vote corrects Tree 3's error.

Summary Table:

Method	How it Handles Noise
Single Decision Tree	Sensitive — one outlier can change the whole tree
Random Forest (Ensemble)	Robust — noise affects at most a few trees; majority vote corrects errors

Outcome: Ensemble learning produces cleaner, more trustworthy models for messy real-world data.